main.c

Share Run

```c
        }
        else if (neighbor != parent) {
            return 1;
        }
        temp = temp->next;
    }
    return 0;
}
int detect_cycle(struct Graph* graph) {
    for (int i = 0; i < graph->numVertices; i++) {
        if (!graph->visited[i]) {
            if (isCycleRecursive(graph, i, -1))
                return 1;
        }
    }
    return 0;
}

int main() {
    struct Graph* graph = createGraph(4);
    addEdge(graph, 0, 1);
    addEdge(graph, 1, 2);
    addEdge(graph, 2, 0);
    addEdge(graph, 2, 3);

    if (detect_cycle(graph)) {
        printf("True: Cycle detected in the graph!\n");
    } else {
        printf("False: No cycle detected.\n");
    }

    return 0;
}
```

Output

Clear

```
True: Cycle detected in the graph!


=== Code Execution Successful ===
```

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct Node {
    int vertex;
    struct Node* next;
};

struct Graph {
    int numVertices;
    struct Node** adjLists;
    int* visited;
};
struct Node* createNode(int v) {
    struct Node* newNode = malloc(sizeof(struct Node));
    newNode->vertex = v;
    newNode->next = NULL;
    return newNode;
}

struct Graph* createGraph(int vertices) {
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct Node*));
    graph->visited = malloc(vertices * sizeof(int));

    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
        graph->visited[i] = 0;
    }
    return graph;
}

void addEdge(struct Graph* graph, int src, int dest) {
```

Output

Clear

```
True: Cycle detected in the graph!


=== Code Execution Successful ===
```

main.c

Share Run

```c
struct Graph* createGraph(int vertices) {
    struct Graph* graph = malloc(sizeof(struct Graph));
    graph->numVertices = vertices;
    graph->adjLists = malloc(vertices * sizeof(struct Node*));
    graph->visited = malloc(vertices * sizeof(int));

    for (int i = 0; i < vertices; i++) {
        graph->adjLists[i] = NULL;
        graph->visited[i] = 0;
    }
    return graph;
}

void addEdge(struct Graph* graph, int src, int dest) {
    struct Node* newNode = createNode(dest);
    newNode->next = graph->adjLists[src];
    graph->adjLists[src] = newNode;

    newNode = createNode(src);
    newNode->next = graph->adjLists[dest];
    graph->adjLists[dest] = newNode;
}
int isCycleRecursive(struct Graph* graph, int v, int parent) {
    graph->visited[v] = 1;

    struct Node* temp = graph->adjLists[v];
    while (temp != NULL) {
        int neighbor = temp->vertex;
        if (!graph->visited[neighbor]) {
            if (isCycleRecursive(graph, neighbor, v))
                return 1;
        }
        else if (neighbor != parent) {
```

Output

Clear

```
True: Cycle detected in the graph!


=== Code Execution Successful ===
```